EzMonitor: Lightweight Metrics Monitoring System

DBMS Course Project Mid-Sem Submission

Team 4 (P1)
Utkarsh Gopal Bhartariya (2024A4PS0656H)
Harshavardhan Marmamula (2024A4PS0560H)
Sanjay Varghese Sunil (2024A4PS0623H)
Diveshraju M (2024A4PS0727H)
Asnani Aayush Pratap (2024A4PS0757H)

October 11, 2026

GitHub Repository: https://github.com/ugeebee/EzMonitor

Part I: Database Design Document

---

1. Problem Statement and Scope

Engineering teams require a reliable method to collect numeric time-series measurements from their infrastructure and receive notifications when metrics cross critical thresholds. EzMonitor is a multi-tenant, relational metrics monitoring system where state components (organizations, instances, time-series data, alert rules, alert history, and notification webhooks) reside in a unified PostgreSQL database. Consistency rules and relationship boundaries are enforced directly by the DBMS rather than application code. The scope encompasses the data model, role-based access control, and alerting workflows, avoiding the overhead of specialized NoSQL engines.

2. Intended Users

- **Admin:** Manages group membership, instances, webhooks, and threshold alert rules.
- **Editor:** Defines and modifies alert rules and instance labels.
- **Viewer:** Reads instance health and alert logs.
- **Scraper / Agent (System Actor):** Pushes scraped metrics into the database through a high-frequency ingest workflow.

3. Important Application Functionality

- **Register Instances:** Manage monitored instances (label, status, creation date) under a specific group; enable or disable them.
- **Ingest Scrape Results:** A batch of metric measurements (CPU, GPU, RAM, Uptime, Network) is stored atomically.
- **Alerting:** Define threshold rules (metric, comparator, threshold). An evaluation pass triggers new alerts and resolves inactive ones; history is maintained for reporting.
- **Notifications:** Dispatch triggered alerts to external endpoints via configured webhooks linked to specific groups and roles.

4. Assumptions

- A sample is recorded at a specific instant.
- Authentication is handled via username/password hash; authorization is explicitly modeled in the database schema via user-group roles.
- Every alert rule applies to all instances of its targeted group.
- Samples older than the retention horizon may be dropped by an external scheduled job; retention policy is not modeled in the schema.

5. Business Rules and Constraints

- **BR1.** Emails and user IDs are unique; email formats are validated.
- **BR2.** A user can belong to many groups with exactly one role per group.
- **BR3.** Within a group, instance labels are unique.
- **BR4.** A time-series sample is unique per (instance ID, timestamp); re-delivered samples overwrite old values.
- **BR5.** A sample value must be positive; samples are rejected for inactive instances.
- **BR6.** An alert rule's webhook notification channel must belong to the same group.
- **BR7.** At most one active firing alert per (rule, instance); a firing alert has an 'Active' status; resolved alerts are never re-opened.
- **BR8.** Deleting a group removes its instances, rules, and webhooks.

6. ER / EER Diagram

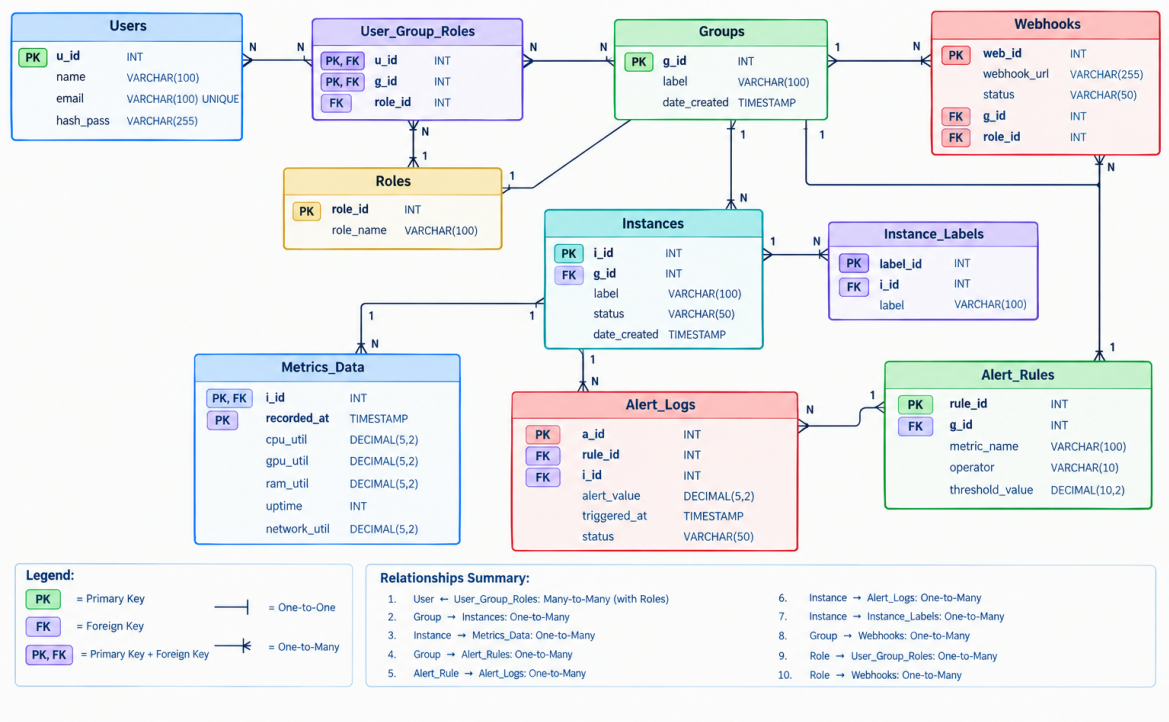


Figure 1: EzMonitor 10-Table Relational Schema ER Diagram

- **Entity Kinds:** `User_Group_Roles` acts as an associative entity resolving the M:N relationship between Users and Groups, carrying the relationship attribute `role_id`. `Metrics_Data` is a weak entity dependent on `Instances` with a partial key of `recorded_at`.
- **Cardinalities:** Groups 1:N Instances / Alert_Rules / Webhooks. Instances 1:N Metrics_Data / Alert_Logs / Instance_Labels.

7. Relational Schema (mapped from the ER/EER model)

Underlined = primary key, italic → R = foreign key referencing R.

Relation	Attributes
users	(u_id, name, email, password_hash) *Candidate keys: email*
groups	(g_id, label, date_created)
roles	(role_id, role_name)
user_group_roles	(*u_id* → users, *g_id* → groups, *role_id* → roles) *M:N resolution; PK is the pair*
instances	(i_id, *g_id* → groups, label, status, date_created)
instance_labels	(label_id, *i_id* → instances, label)
metrics_data	(*i_id* → instances, recorded_at, cpu_util, gpu_util, ram_util, uptime, network_util) *Weak entity: PK = owner key + partial key recorded_at*
alert_rules	(rule_id, *g_id* → groups, metric_name, operator, threshold_value)
alert_logs	(a_id, *rule_id* → alert_rules, *i_id* → instances, alert_value, triggered_at, status)
webhooks	(web_id, webhook_url, status, *g_id* → groups, *role_id* → roles)

- **ER → relational mapping rules applied:** Strong entity relations utilize surrogate integer keys. 1:N relationships apply a FK on the N side. The weak entity `metrics_data` uses a composite PK encompassing the owner's PK and a partial timestamp key, with ON DELETE CASCADE.
- **Domains and data types:** Surrogate keys utilize INT/SERIAL. Time utilizes TIMESTAMP. Measurements utilize DECIMAL. Flags and statuses utilize VARCHAR + CHECK.

8. Primary / Foreign Keys and Referential Actions

Foreign key	References	ON DELETE	Reason
user_group_roles.g_id	groups	CASCADE	membership is meaningless without the group
user_group_roles.u_id	users	CASCADE	same, for the user
instances.g_id	groups	CASCADE	a group owns its assets; deleting the tenant removes them
alert_rules.g_id	groups	CASCADE	tenant deletion removes associated rules
webhooks.g_id	groups	CASCADE	tenant deletion removes notification channels
metrics_data.i_id	instances	CASCADE	weak entity follows its owner
instance_labels.i_id	instances	CASCADE	weak entity follows its owner
alert_logs.rule_id	alert_rules	CASCADE	history belongs to the rule
alert_logs.i_id	instances	CASCADE	history belongs to the instance

9. Integrity Constraints Enforced by the DBMS

Mechanism	Where/what
NOT NULL	All attributes except documented optional ones.
UNIQUE	users(email).
CHECK (domain)	status enumerations; threshold_value and metric values must be ≥ 0.
CHECK (row logic)	alerts lifecycle: an alert must be 'Active' or 'Resolved'.
Trigger T1	BEFORE INSERT on metrics_data: reject samples for inactive instances.

10. Normalization Analysis

All relations have been normalized to the Third Normal Form (3NF). There are no partial dependencies (satisfying 2NF) and no transitive dependencies between non-key attributes (satisfying 3NF).

Relation	Functional dependencies	NF
users	u_id → all; email → all	3NF
groups / roles	id → all	3NF
user_group_roles	(u_id, g_id) → role_id	3NF
instances	i_id → all	3NF
instance_labels	label_id → all	3NF
metrics_data	(i_id, recorded_at) → cpu, gpu, ram, uptime, net	3NF
alert_rules / webhooks	id → all	3NF
alert_logs	a_id → all	3NF

11. Design Justification and Key Decisions

- **Natural composite key for samples:** `(i_id, recorded_at)` forbids duplicate points, establishes an idempotent upsert for ingest, and naturally clusters an instance's range scans on the primary index.
- **Alerts are separate from rules:** A rule represents configuration, whereas alerts represent history (many per rule over time). This strict separation prevents redundant storage of operator logic.
- **Role-Based Access Control (RBAC):** The `user_group_roles` relation strictly isolates multi-tenant access, guaranteeing users can safely participate in multiple groups with varying permissions without data duplication.

Part II: Application Design Document

1. Technology Stack

The overall system architecture follows a Front End → Backend → Relational Database pipeline to ensure separation of concerns and scalable data management.

- **Database Tier:** PostgreSQL (Version 16+). PostgreSQL was selected for its robust ACID compliance and advanced indexing capabilities.
- **Backend / Application Layer:** Go. The backend leverages Go's concurrency model (goroutines) to efficiently handle high-throughput, high-frequency metric ingestion streams from multiple instances without blocking.
- **Frontend Interface:** React (with TypeScript) and Vite/Next.Js. The frontend provides a responsive visualization layer, utilizing charting libraries like Recharts or Grafana-UI components to render time-series aggregations.
- **Deployment & Infrastructure:** Docker containers for isolated service environments, orchestrated via Docker Compose. Caddy serves as the reverse proxy, handling automatic SSL termination and routing frontend API requests to the Go backend.

2. High-Level Application Architecture

The architecture is designed to decouple the high-volume metric ingestion path from the complex read-heavy query path used by the dashboards.

1. **Data Producers (Scrapers/Agents):** Lightweight daemons running on monitored host instances collect local system metrics (CPU, GPU, RAM, Uptime, Network) at configurable intervals (e.g., every 15 seconds) and push them over HTTP/REST to the ingest API.
2. **Ingestion & Buffering Pipeline:** The Go backend receives the JSON payloads, authenticates the instance using its registration token and executes bulk SQL `INSERT` transactions against the PostgreSQL database.
3. **Relational State Management:** PostgreSQL acts as the single source of truth. It stores not only the time-series fact data (`metrics_data`) but also the entire configuration state, including Users, Groups, Roles, Dashboards, and Webhooks.
4. **Query & Alerting Engine:**
   - *Alerting:* Implemented primarily at the database level using PostgreSQL triggers that fire upon new data insertion, evaluating values against the `alert_rules` table.
   - *Querying:* The backend exposes REST API endpoints that translate frontend dashboard requests into complex SQL queries using Common Table Expressions (CTEs) and time-bucketing functions.
5. **Visualization Client:** The React frontend consumes the query API, enforcing Role-Based Access Control to ensure users only view dashboards and instances associated with their assigned Groups.

3. Proposed Application Workflows

To thoroughly exercise the relational database, the application implements three distinct end-to-end workflows, each demonstrating different DBMS concepts.

Workflow 1: High-Frequency Metric Ingestion (Transaction & Constraint Enforcement)
This workflow demonstrates coordinated changes and strict integrity enforcement.

- **Action:** A monitored instance transmits an array of its latest hardware metrics to the backend.
- **Execution:** The Go backend initiates a database transaction. It first validates that the instance's `i_id` exists and is marked as 'Active' in the `instances` table. It then attempts to batch-insert the records into the `metrics_data` table.
- **DBMS Integration:** If a metric value is negative, it violates a `CHECK` constraint. If the instance is unregistered, it violates a `FOREIGN KEY` constraint. Any violation causes the entire batch transaction to roll back, preventing partial data corruption.

Workflow 2: Dynamic Dashboard Aggregation (Complex Retrieval & Grouping)
This workflow focuses on advanced data retrieval, searching, and aggregation.

- **Action:** A Group Admin logs into the React frontend and opens the "Global Infrastructure Health" dashboard, set to view the last 24 hours of data.
- **Execution:** The frontend requests the data window from the Go API. The backend constructs a complex query utilizing `JOIN`s across `groups`, `instances`, `instance_labels`, and `metrics_data`.
- **DBMS Integration:** The SQL query uses Common Table Expressions (CTEs) to filter instances by specific JSONB labels. It then applies window functions and aggregate functions (e.g., `AVG()`, `MAX()`) grouped by 5-minute time buckets (`DATE_TRUNC`). This offloads the heavy analytical processing to PostgreSQL, returning only the final, lightweight dataset to the frontend for charting.

Workflow 3: Automated Alert Evaluation & Dispatch (Database Triggers)
This workflow demonstrates automated state transitions and the use of database-level triggers.

- **Action:** A database node's CPU utilization spikes to 98%, crossing a critical threshold defined by the engineering team.
- **Execution:** As the ingest workflow inserts the new 98% reading into `metrics_data`, a `BEFORE INSERT` database trigger automatically fires.
- **DBMS Integration:** The trigger invokes a stored procedure that compares the incoming value against the `alert_rules` associated with that instance's group. Detecting a breach, the procedure generates a new record in the `alert_logs` table with a status of 'Active'. The Go backend continuously polls (or listens via Postgres `LISTEN/NOTIFY`) for new active alerts, joining them with the `webhooks` table to resolve the destination URL, and dispatches the critical notification payload to the team's Slack or Discord channel.